

Publication & Patent Guidance — Read This Before You Do Anything With Documents 1-3

The short version

- **Publish freely** — documents 1-3 in this folder are written to be public, and pushing them in this PR is consistent with that.
- **Do not try to patent anything in this repository right now.** None of it is novel, and I mean that in the specific legal sense the word has in patent law, not as a value judgment.
- **If you ever do have something patentable, the single most important rule is: file before you disclose, not after.** This PR is a public disclosure. That is fine for documents 1-3 and would be fatal for a patent application on the same content.

1. Why nothing here is patentable

Patentability (in essentially every jurisdiction, under both US and non-US law) requires the claimed invention to be **novel** and **non-obvious** over the prior art. Walking through what actually exists in this repository against that bar:

- **The 20 landed fixes are corrections, not inventions.** Every one of them makes the code do what its own docstring, comment, or established sibling code already said it should do. "The code claimed to validate a hostname and didn't" is not an invention when fixed — restoring documented intended behavior is, definitionally, not new. A patent examiner would reject every one of these on prior art grounds within minutes: hostname-boundary validation, XPath injection remediation via `concat()`, CDP object-group release, exception handling on a truncated LLM completion — all of these are described in public prior art that predates this repository by years (see the references in documents 2 and 3).
- **The four ideas-backlog entries are explicitly derived from a named competitor's already-public technique.** Check `docs/agent-notes/ideas-backlog.json` — every `originating_agent` field says, verbatim, "from reading agentrhq/webcmd." You cannot patent something you read in someone else's public README. If any of those four ideas were ever built into something genuinely differentiated (not "we also do snapshot pruning" but a specific, non-obvious *mechanism* for doing it that isn't what webcmd or any other prior art already does), *that specific mechanism* — not the general idea of "prune the snapshot" — might be worth a real novelty search. None of them have been built yet; they're all still `proposed` or `parked`.
- **The multi-agent verification methodology (document 1) is a case study of applying existing techniques, not a new one.** It composes self-refinement (Madaan et al. 2023), adversarial multi-agent debate (Du et al. 2023), and constitutional-AI-style critique (Bai et al. 2022) — all separately published

years before this session existed. Combining known techniques can occasionally be patentable if the combination itself is non-obvious and produces an unexpected result, but that is a genuinely hard bar, it requires a real prior-art search by someone qualified to do one, and "we ran an existing pattern against a browser-automation codebase" is a straightforward, foreseeable application, not an unexpected one.

2. If something here ever *does* become patentable

This would require: (a) someone actually builds one of the parked ideas into a working, specific, non-obvious mechanism, and (b) a real novelty search turns up nothing anticipating that specific mechanism. If both of those become true:

1. **Stop. Do not commit or push it publicly first.** Filing timing: - The US is **first-inventor-to-file** with a narrow (12-month) grace period for the inventor's *own* prior public disclosure — but that grace period does not exist in most of the rest of the world. - The EU, and most other major jurisdictions, use **absolute novelty**: *any* public disclosure before filing — including a GitHub commit, a blog post, or a conference talk, by anyone, including you — destroys patentability outright, with no grace period. - Practical rule: treat every jurisdiction as absolute-novelty and file first, always. There is no upside to disclosing before filing and a catastrophic, irreversible downside if you guess wrong about which countries matter to you later.
2. **A provisional application is the fast, cheap first step**, and — this matters — you (the inventor) are legally permitted to file it yourself without an attorney: - Official USPTO overview: <https://www.uspto.gov/patents/basics/apply/provisional-application> - Filed electronically through USPTO Patent Center (you'll need an account). - Fees (2026, per the USPTO's own schedule): \$325 (large entity), \$130 (small entity), \$65 (micro entity) for the provisional filing itself. - A provisional buys you exactly 12 months to file the real (non-provisional) application claiming its priority date — miss that window and the provisional's priority date is lost.
3. **Get a registered patent attorney or agent for anything beyond the provisional**, and ideally for the provisional's claim language too — the USPTO explicitly does not check whether a self-filed provisional is complete or strong enough to support later claims; a vague or incomplete provisional can quietly fail to protect the invention it was meant to cover. - Find one registered to practice before the USPTO (this is a real, separate bar exam from the general practice of law — not every lawyer qualifies): <https://www.uspto.gov> (search "registered patent attorney" or use the USPTO's practitioner search tool linked from that site). - I am not a patent attorney, this is not legal advice, and I will not draft claim language — defective claims can be worse than no patent at all (unenforceable, or narrower than the actual invention), and that determination needs a professional, not an LLM.

3. Where documents 1-3 could honestly go (real venues, real links)

Document 1 (methodology case study): - An engineering blog post — the most natural fit; case studies like this don't need peer review to be useful, and the honest "what is not established" section (§5 of that document) is exactly the kind of content a peer reviewer would otherwise demand you add. - arXiv, if you want a citable, timestamped preprint: register at <https://arxiv.org/user/register>, submit as a ZIP of LaTeX sources (arXiv does not accept plain Markdown), pick a category from https://arxiv.org/category_taxonomy (likely `cs.SE` or `cs.AI`). **Note:** first-time submitters to some categories need an endorsement from an existing arXiv author in that category — factor in that step's latency. Submission guide: https://info.arxiv.org/help/submit_index.html - A workshop on LLM agents/tooling (venues rotate year to year — search current CFPs close to when you're ready rather than trusting a link here to still be live).

Document 2 (security advisory): this one has exactly one honest home — this repository's own GitHub Security Advisories, per the concrete steps already written into document 2 §7. Do not publish a security advisory anywhere else first; the standard, expected practice is to disclose to the maintainer/repository first (which, since this is your repository, is already satisfied) and request a CVE through the advisory itself rather than through a third party.

Document 3 (case studies): same options as document 1 — an engineering blog is the natural home; these are exactly the kind of concrete, narrowly-scoped writeups that get shared well in that format precisely because each one names a real, checkable defect class rather than making a general claim.

4. The one-paragraph version to remember

Publish the writeups; they're honest, they're useful, and public is where they belong. Don't let anyone (including a future version of this project) call a bug fix an invention, and don't let "we should patent this" happen after something has already been pushed to a public branch — by then it's not a decision anymore, it's already been made for you.